

Числа, системи числення, математична логіка

1. Способи представлення нецілих чисел

У комп'ютері дробові числа можуть зберігатися двома основними способами:

1. у форматі з фіксованою комою,
2. у форматі з плаваючою комою.

1.1. Представлення з фіксованою комою

Число подається так, що частина бітів відводиться під цілу частину, а частина — під дробову.

Наприклад, 16-бітне представлення може містити 8 біт цілої частини і 8 біт дробової.

Точність фіксована і не змінюється.

Такий спосіб часто використовують у мікроконтролерах, DSP-системах, у фінансових задачах, де потрібна чітка фіксована точність.

Приклад: 5.75 у двійковому вигляді має вигляд 101.11, де 101 — ціла частина, 11 — дробова.

1.2. Представлення з плаваючою комою

У цьому випадку число подається у вигляді:

$$(-1)^s \times 1.m \times 2^e$$

де s — знак числа,

m — мантиса (дробова частина),

e — порядок степеня двійки.

Перевага цього формату в тому, що він дозволяє представляти дуже великі і дуже маленькі числа.

Основні формати:

Формат float (32 біти):

1 біт — знак

8 біт — порядок

23 біти — мантиса

Точність приблизно 7 десяткових цифр.
Діапазон приблизно від 10^{-38} до 10^{38} .

Формат double (64 біти):

1 біт — знак

11 біт — порядок

52 біти — мантиса

Точність приблизно 15–16 десяткових цифр.
Діапазон приблизно від 10^{-308} до 10^{308} .

2. Особливості та проблеми чисел з плаваючою комою

- Деякі десяткові дроби неможливо точно подати у двійковому вигляді.
Наприклад, десятковий дріб 0.1 у двійковому вигляді є нескінченним періодичним дробом.
У результаті значення зберігається лише приблизно.
- В обчисленнях накопичуються похибки, бо результат кожної операції часто округлюється.
- Формат IEEE-754 має спеціальні значення:
плюс нескінченність, мінус нескінченність, NaN (не число).

3. Приклад представлення числа 5.75 у форматі float (32 біти)

- Записуємо число у двійковій системі:
 $5 = 101$
 $0.75 = 0.11$
Число 5.75 має вигляд 101.11.
- Потрібно нормалізувати число до вигляду 1.фракція $\times 2^e$.
 $101.11 = 1.0111 \times 2^2$.
- Обчислюємо порядок.
Для float використовується зміщення 127.
Отже порядок дорівнює $2 + 127 = 129$.
- Мантиса — це частина після крапки у записі 1.0111 (тобто 0111 і далі нулі до довжини 23 біти).

Отримуємо бітові послідовності для знака, порядку та мантиси.

У шістнадцятковому вигляді число 5.75 у форматі float представляється як 40B80000.

4. Представлення числа 5.75 у форматі double (64 біти)

Принцип повністю той самий, тільки з іншим зміщенням та довшою мантиєю.

1. Нормалізована форма зберігається та сама: 1.0111×2^2 .
2. Зміщення для double дорівнює 1023.
Порядок дорівнює $2 + 1023 = 1025$.
3. Мантия містить значення 0111 і 48 нулів після неї (усього 52 біти).

У шістнадцятковому форматі значення дорівнює 4017000000000000.

5. Чому виникає проблема з дробом 0.1

Десятковий дріб 0.1 у двійковому форматі перетворюється на нескінченний періодичний дріб.

У нормалізованому вигляді він виглядає як $1.100110011001... \times 2^{-4}$.

Неможливо зберегти нескінченний двійковий дріб точно, тому зберігається лише наближення.

Через це в пам'яті значення 0.1 насправді є трохи більшим за 0.1.

У форматі float значення 0.1 наближено зберігається як 0.10000000149...

У форматі double як 0.1000000000000000000555...

6. Чому порядок у форматах IEEE-754 зберігається зі зміщенням

Порядок може бути від'ємним (наприклад, для дробів), тому безпосередньо записати його бітами неможливо, бо бітове поле порядку не має знаку.

Тому використовують зміщення (bias):

для float — 127,

для double — 1023.

Це дозволяє кодувати як додатні, так і від'ємні порядки звичайними беззнаковими числами.

7. Порівняння float і double

Формат float займає 32 біти, має меншу точність (близько 7 знаків) і менший діапазон.

Формат double займає 64 біти, має точність 15–16 цифр і значно більший діапазон.

У наукових та інженерних обчисленнях зазвичай використовують double.

У графіці, іграх та системах із обмеженою пам'яттю часто використовують float.

8. Практичні наслідки використання чисел з плаваючою комою

1. Не можна точно порівнювати дроби за допомогою операції «дорівнює».

Наприклад, вираз

$0.1 + 0.2 == 0.3$

повертає false у більшості мов програмування.

2. Для коректного порівняння використовують перевірку з похибкою:
абсолютна різниця між числами повинна бути меншою за деяке дуже мале значення.
3. Якщо потрібна точність у десяткових дробах, часто використовують представлення з фіксованою комою або спеціальні десяткові формати.